

Developer Operations (devops)

Containerization



Introduction, Why docker?

IT WORKS ON MY MACHINE

THEN WE'LL SHIP YOUR MACHINE

AND THAT IS HOW DOCKER WAS BORN

And Users Be Like

```
# our base image
FROM alpine:3.5

# Install python and pip
RUN apk add --update py2-pip

# upgrade pip
RUN pip install --upgrade pip

# install Python modules needed by the Python app
COPY requirements.txt /usr/src/app/
RUN pip install --no-cache-dir -r /usr/src/app/requirements.txt

# copy files required for the app to run
COPY app.py /usr/src/app/
COPY templates/index.html /usr/src/app/templates/

# tell the port number the container should expose
EXPOSE 5000

# run the application
CMD ["python", "/usr/src/app/app.py"]
```

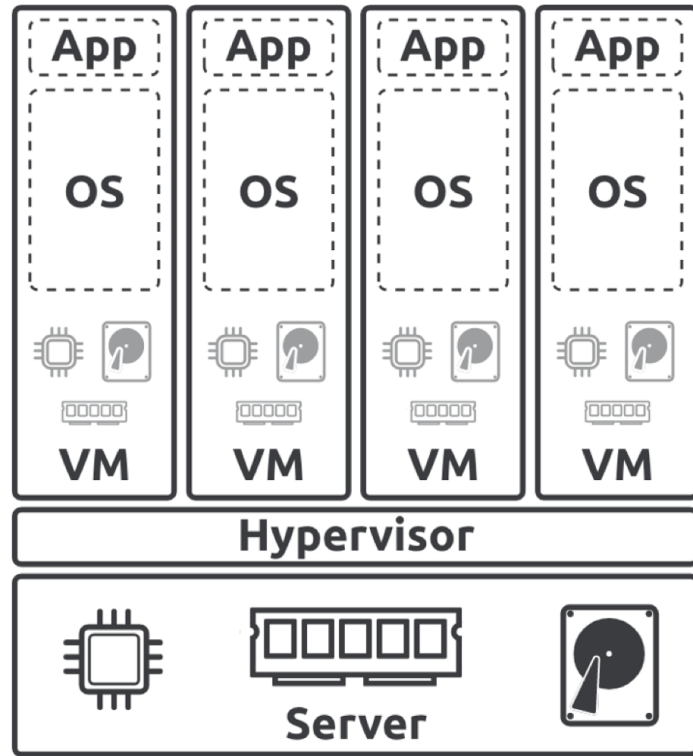
```
# requirements.txt
scikit-learn
pandas
flask
numpy
scipy
```



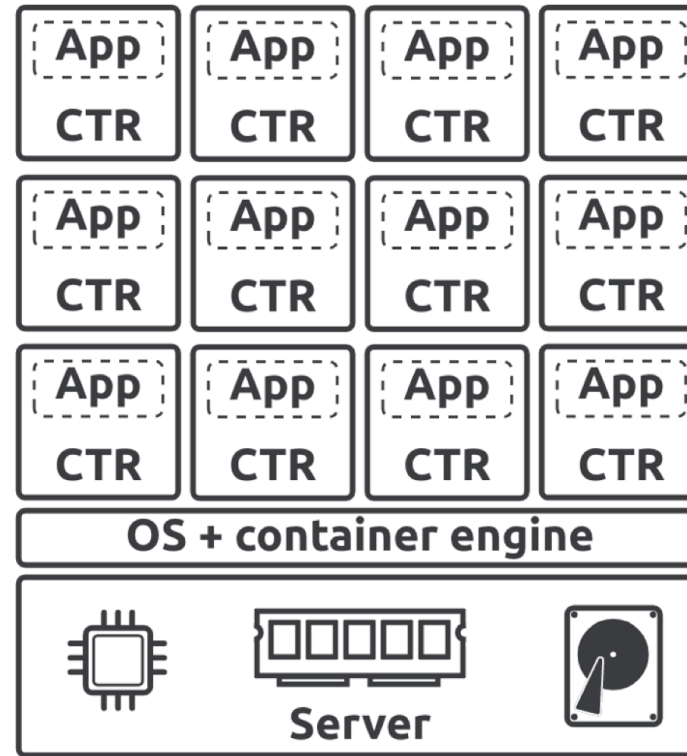

Why do we need Container for developing and operating applications?

① Start presenting to display the poll results on this slide.

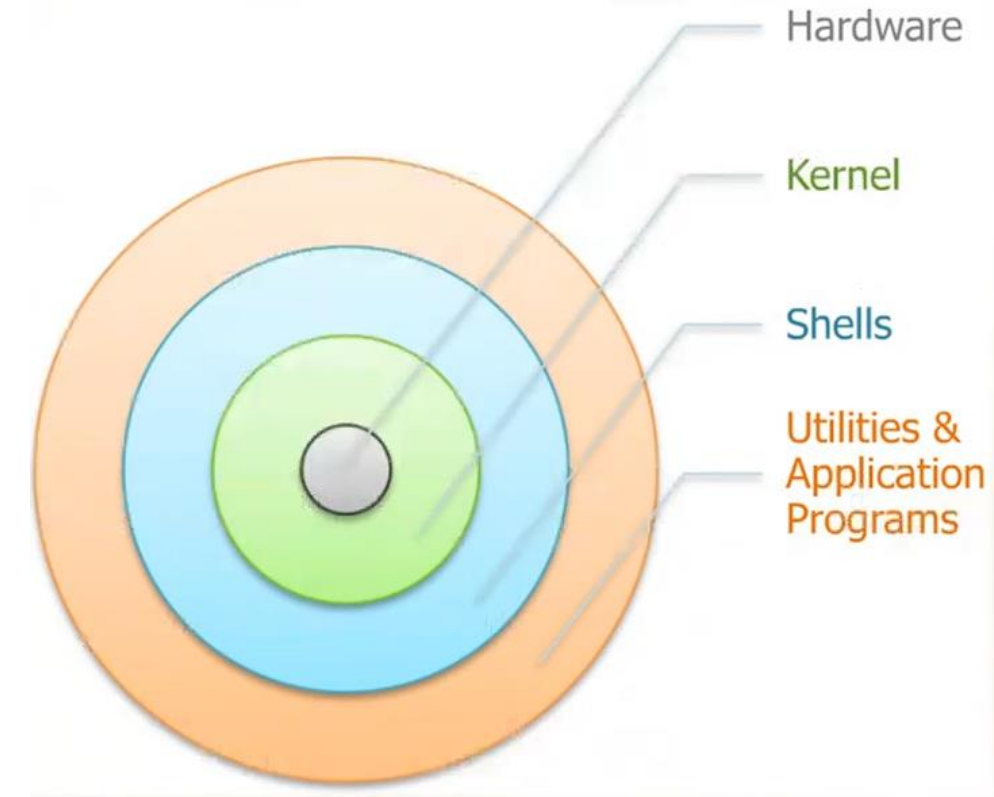
Where does it come from?



VM model



Container model



Clear separation of concerns is mandatory for good operating!
→ See Point 2 of 12Factor-App

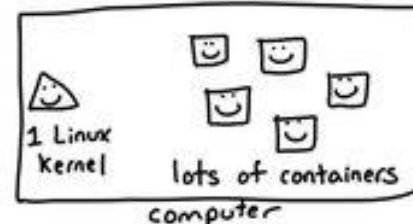
Linking to Cloud, Para-Virtualization



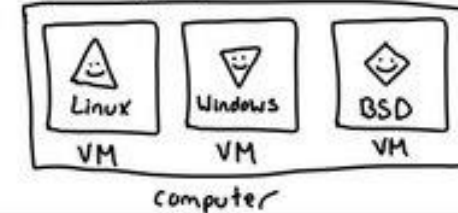
JULIA EVANS
@b0rk

containers vs VMs

a container is a
group of processes

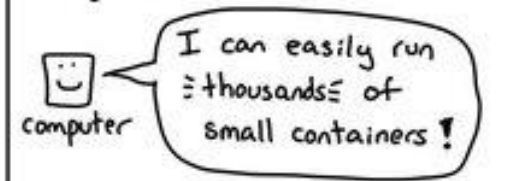


a virtual machine is
a fake computer
each one has its own
operating system!



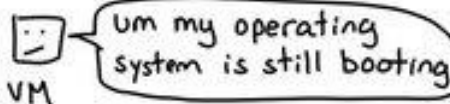
containers use
less RAM

This is because they share a
single Linux kernel.

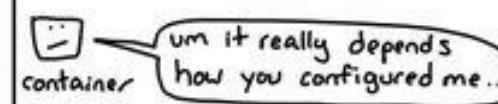
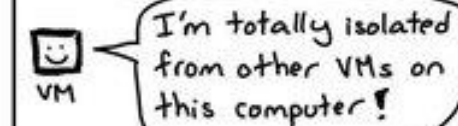


containers start faster

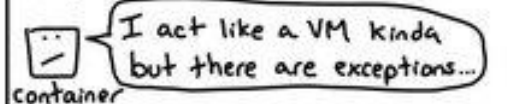
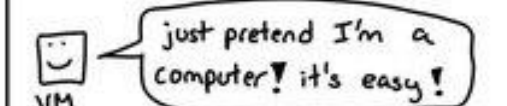
because they're processes and
process start fast ♥



containers are more
complicated to secure



it's harder to figure
out what you can do
in a container

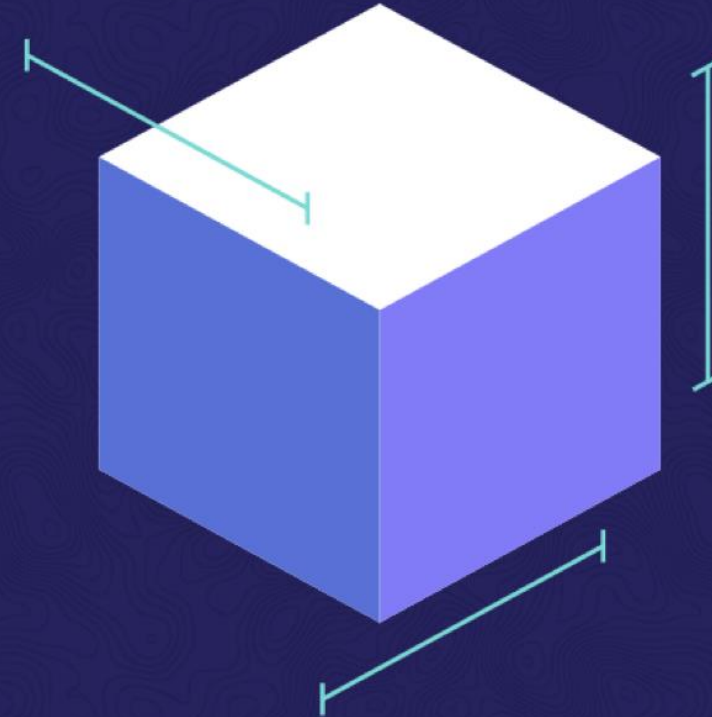


Open Container Initiative

The **Open Container Initiative** is an open governance structure for the express purpose of creating open industry standards around container formats and runtimes.

Established in June 2015 by Docker and other leaders in the container industry, the OCI currently contains two specifications: the Runtime Specification (runtime-spec) and the Image Specification (image-spec). The Runtime Specification outlines how to run a "filesystem bundle" that is unpacked on disk. At a high-level an OCI implementation would download an OCI Image then unpack that image into an OCI Runtime filesystem bundle. At this point the OCI Runtime Bundle would be run by an OCI Runtime.

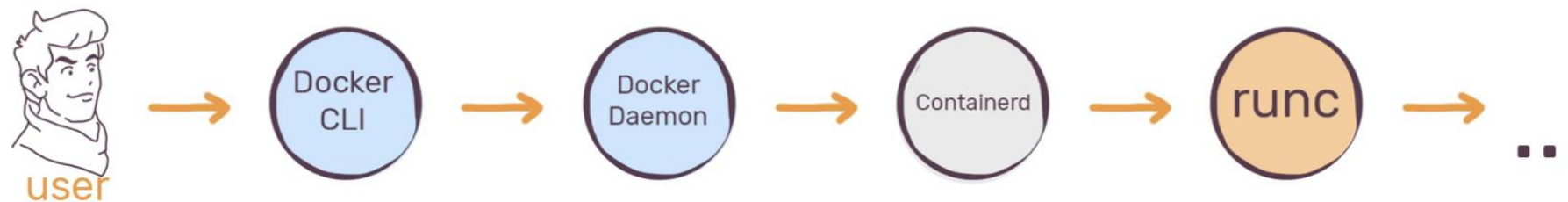
Learn more ↗



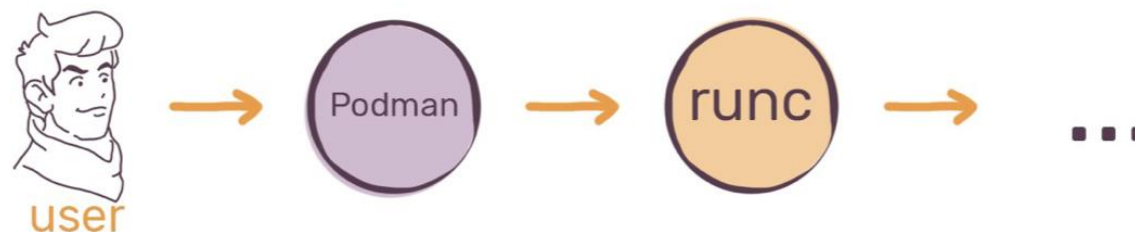
<https://github.com/opencontainers/runtime-spec>

<https://github.com/opencontainers/image-spec>

Entwickler auf
Windows & MacOS



Auf Redhat
Enterprise Linux



Kubernetes selber installiert
Google Cloud/AWS



Kubernetes selber installiert
Google Cloud/AWS



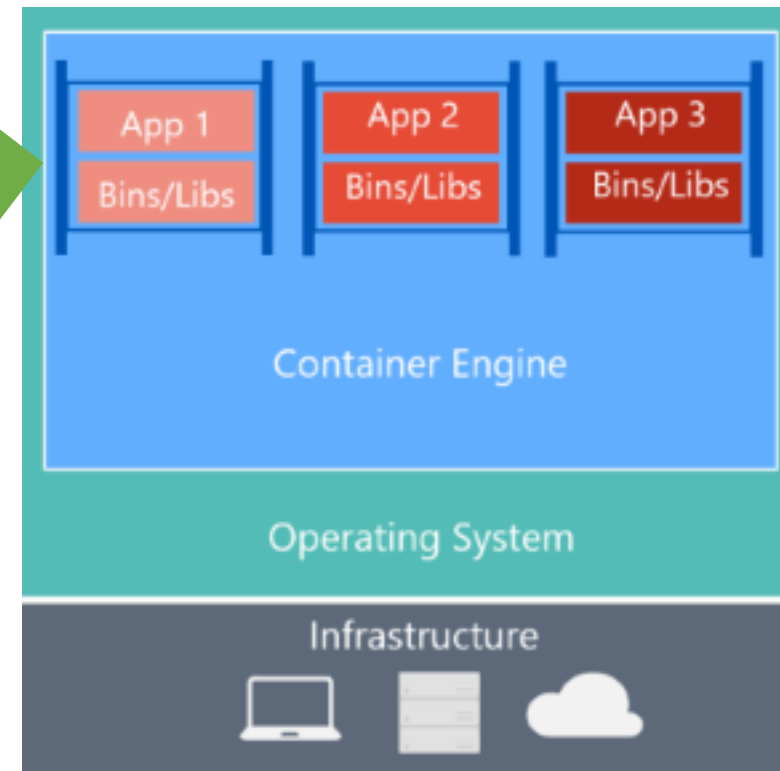
<https://mkdev.me/posts/the-tool-that-really-runs-your-containers-deep-dive-into-runc-and-oci-specifications>

Dockerfile and Dockerimage

```

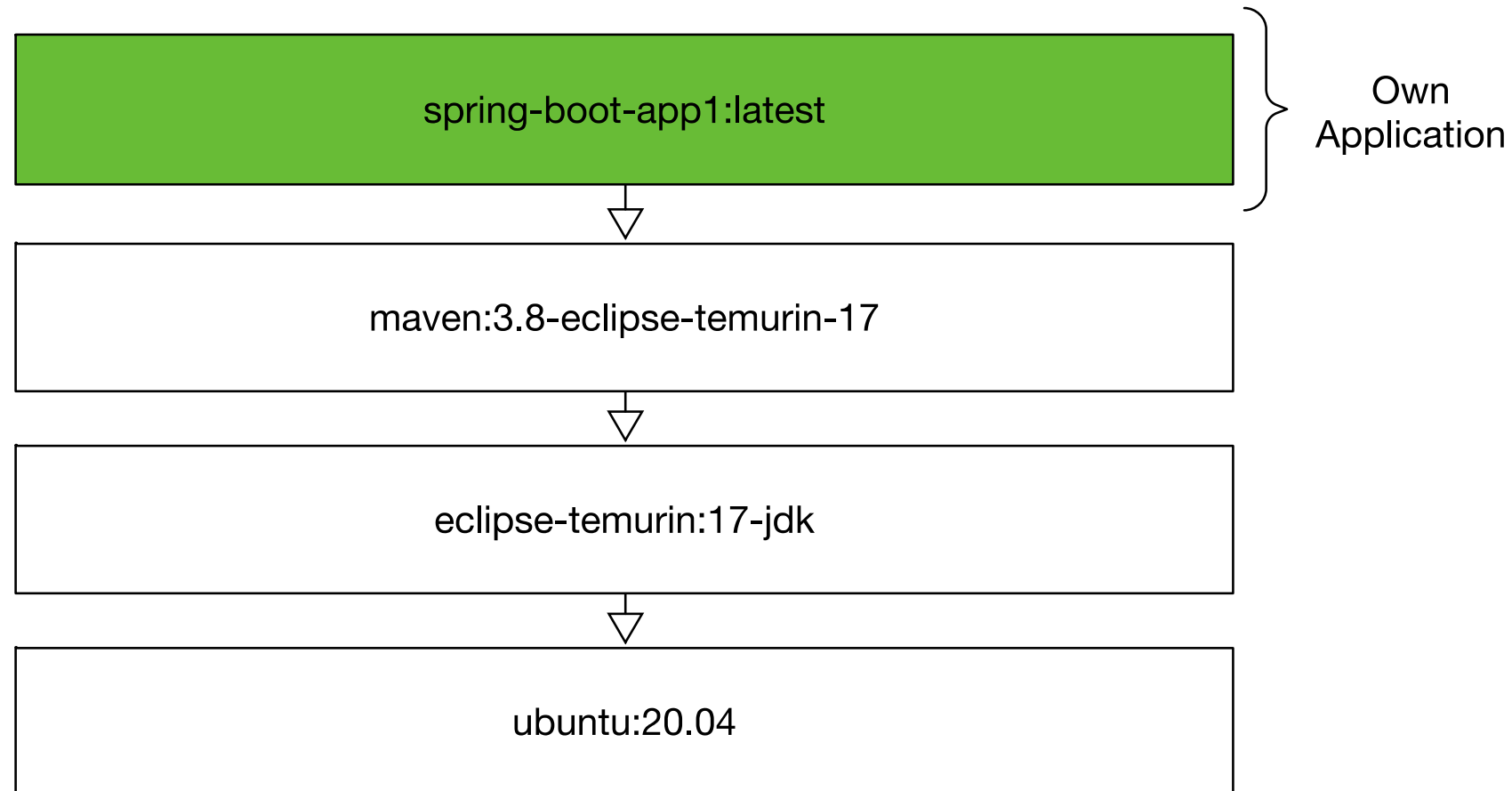
1 FROM ubuntu:15.04
2 COPY . /app
3 RUN make /app
4 CMD python /app/app.py
  
```

Line: 1:5 Dock... | Tab Size: 4

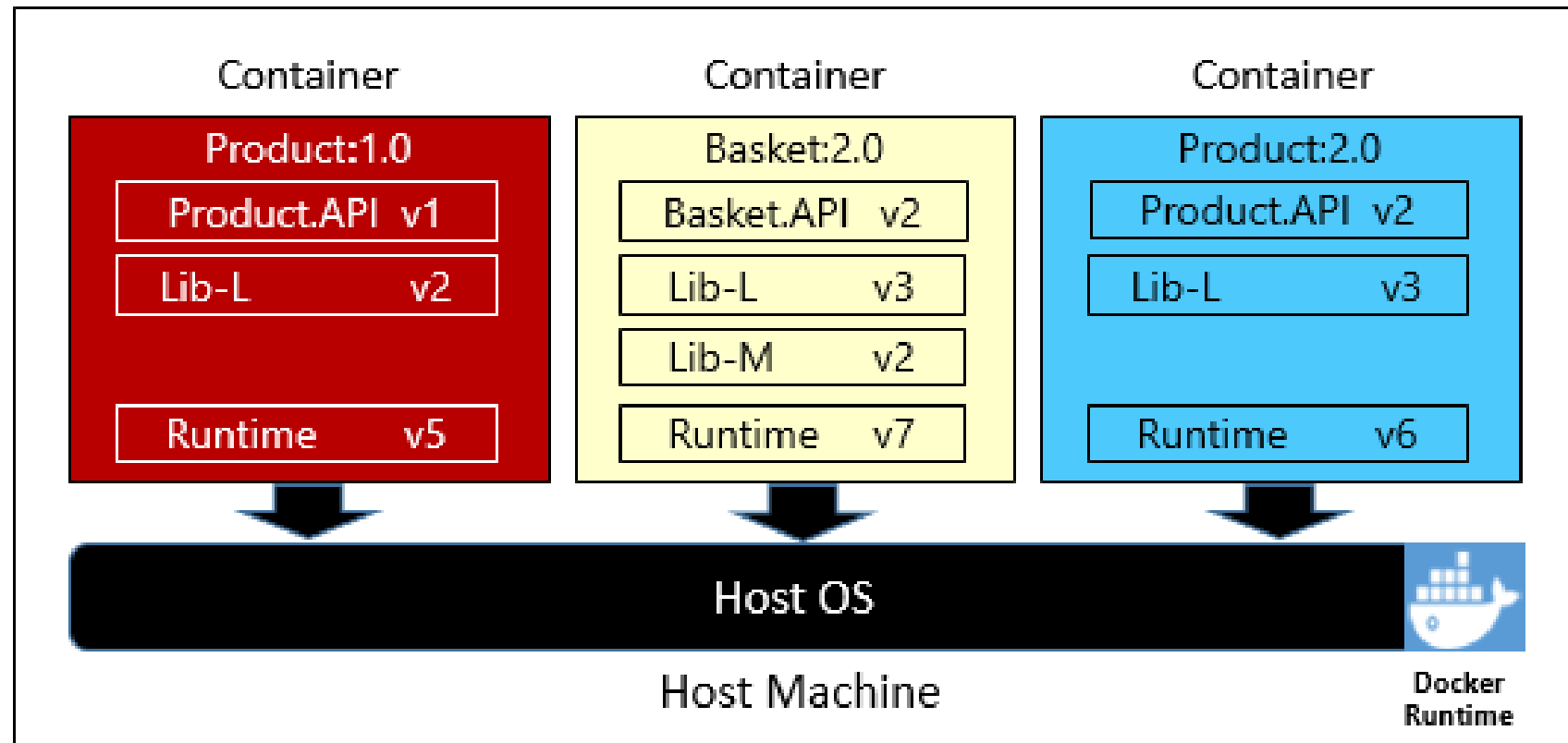


<https://docs.docker.com/storage/storagedriver/images/container-layers.jpg>

Immutable Software



Clear isolation guaranteed



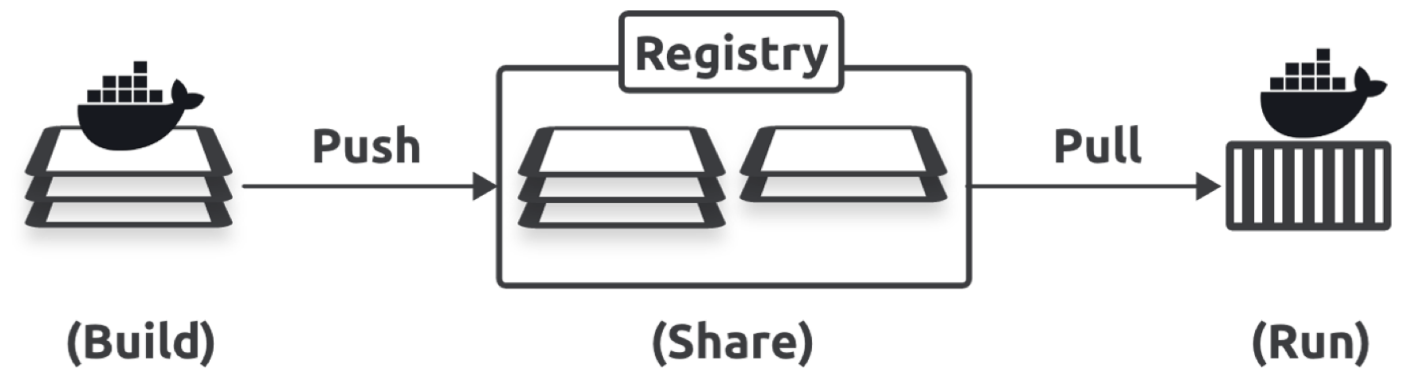
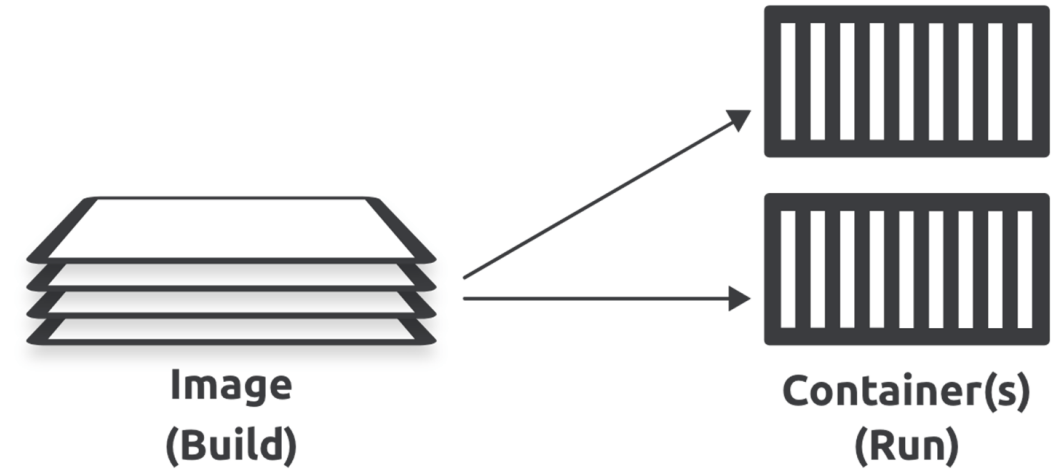
Images and Container

Image:

- Stacked Representation of all packages / software to start a process
- Might be shared to multiple clients / services

Container:

- Process to be started
- Bases on Image
- Has lifecycle of a process

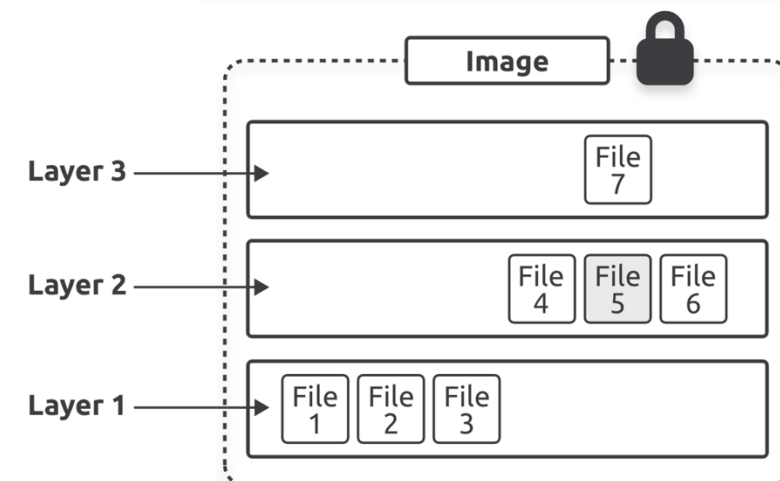
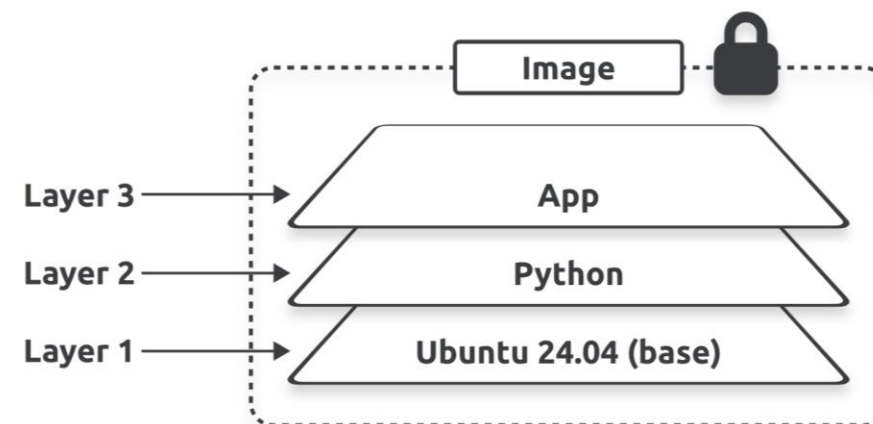


Detailed view of layers

Each layer bases on the former layer:

- Put large elements down in the OCI-image
 - ...so one can leverage from caching in storage and transfer
- Put regular updated content to the upper layers
 - ...so that non-regular updated content can be re-used
- Reflect what you actually need in your image

```
$ docker pull redis:latest
latest: Pulling from library/redis
25d3892798f8: Download complete
e5d458cf0bea: Download complete
4f4fb700ef54: Already exists <----- This line
<Snip>
```



**Merged view of
all three layers**



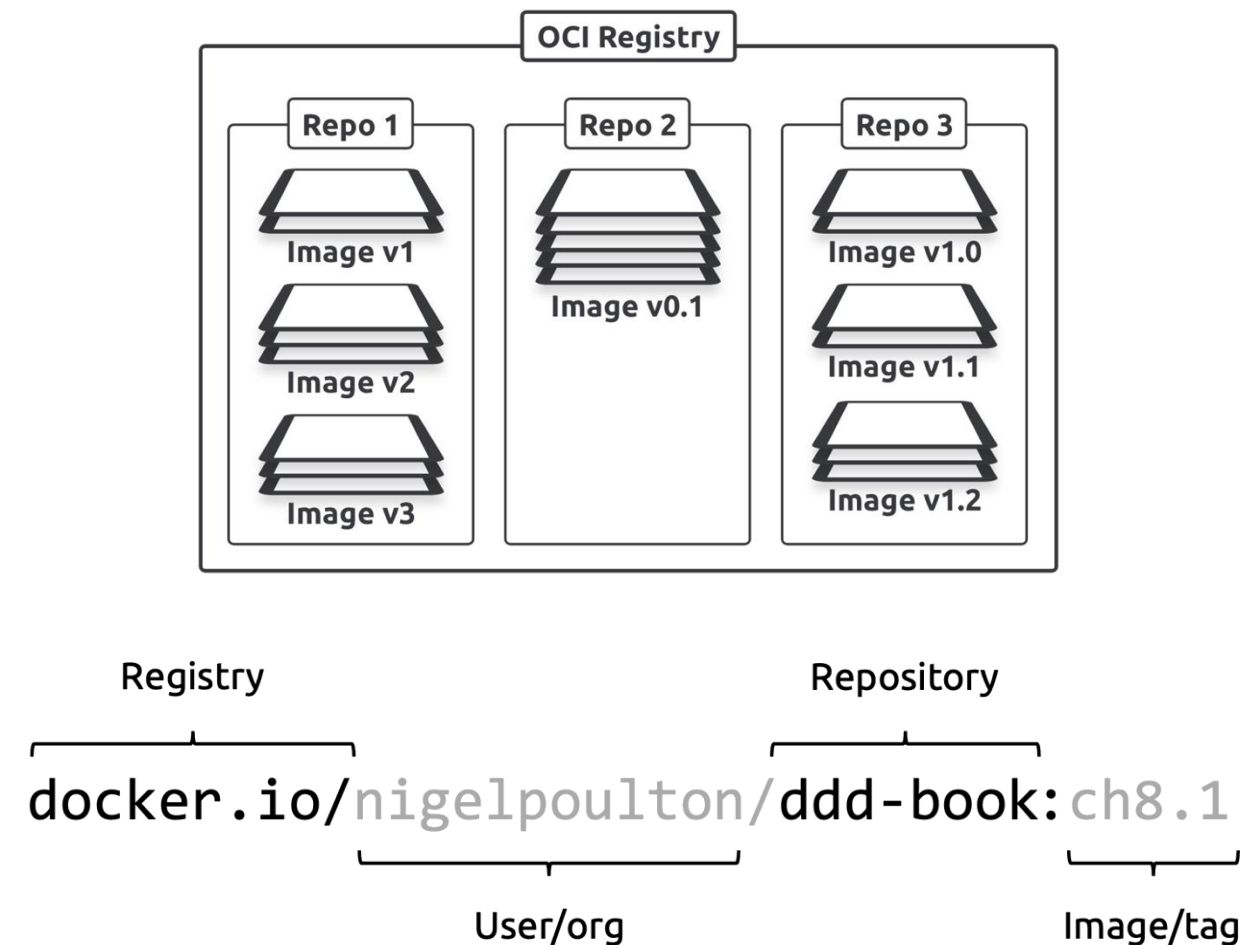
Registries

Images are (commonly) build automatically and stored on a server.

→ Registry

- Can be seen as repository for images
- Accessible via URL
- There are many free registries for common software out there

- Beware of public images!



Out of the image, into the container

Create a container out of a image via:

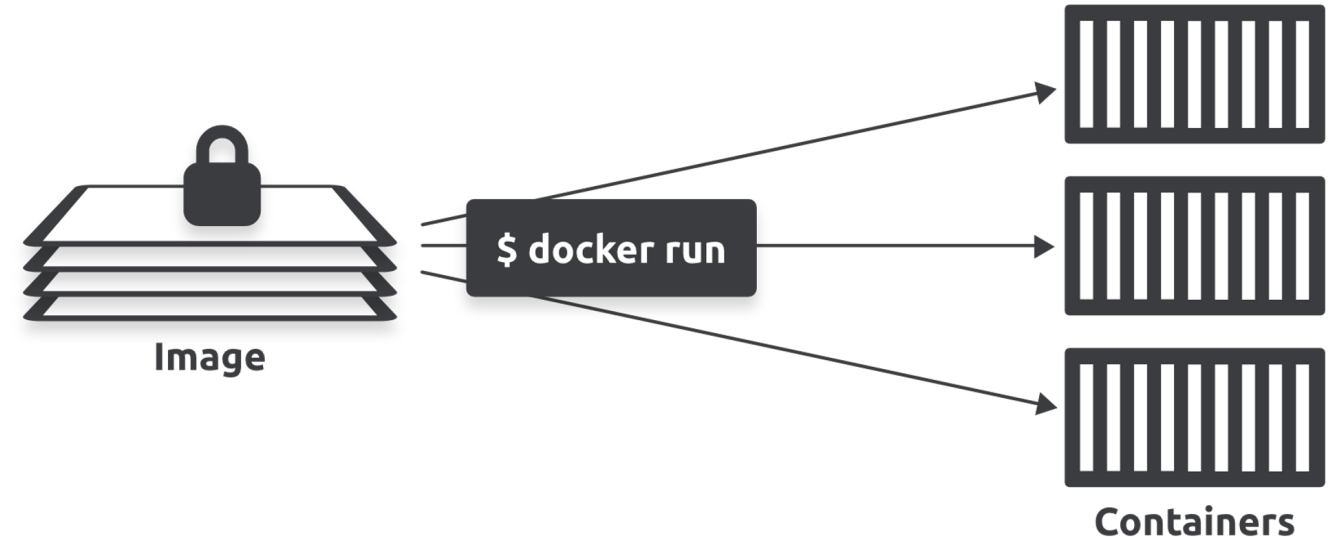
```
docker run IMAGE
```

Be aware that:

- You need port mapping
- You need volume mappings
- There should always be the main program as PID 1

Useful commands:

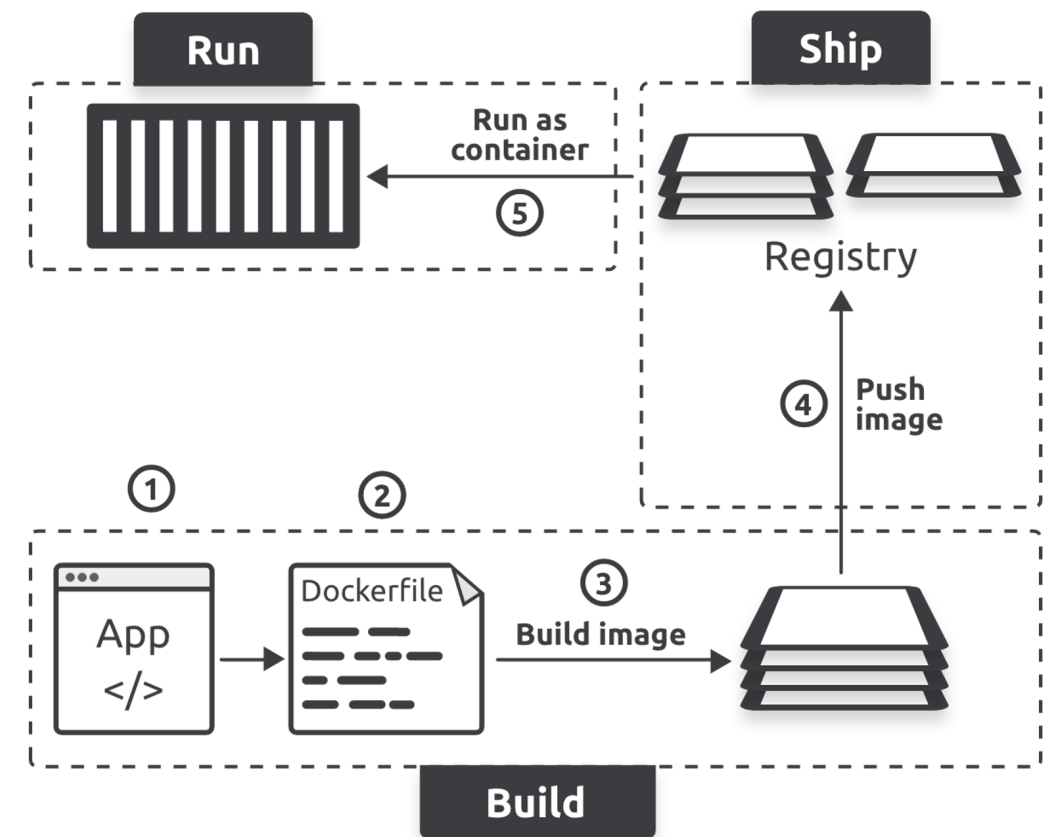
- `docker run` #starting container
- `docker ps` #showing container
- `docker exec` #executing process in container
- `docker inspect` #getting information about container



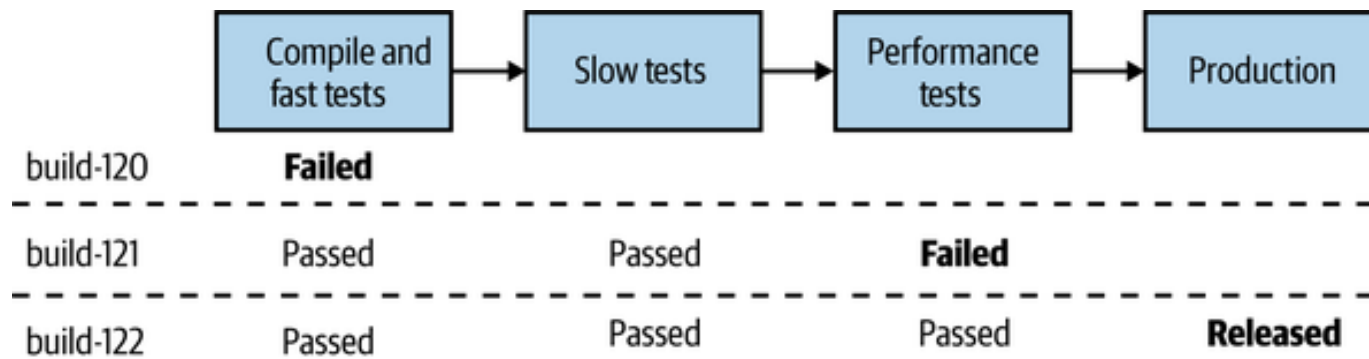
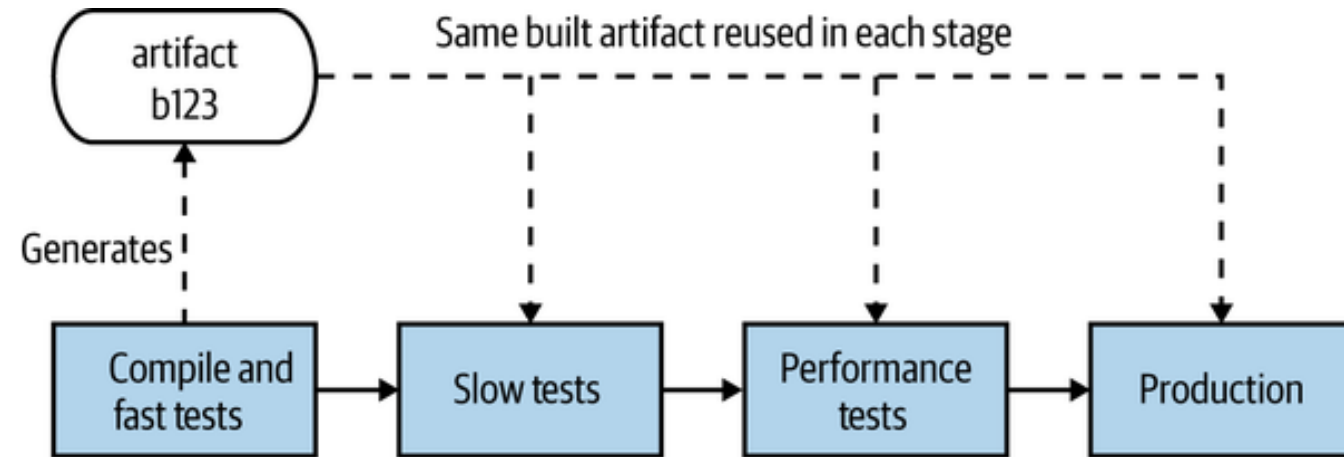
How to bring your application into a container?

e.g.:

1. Write your applications and create the list of dependencies
2. Create a Dockerfile that tells Docker how to build and run the app
3. Build the app into an image
4. Push the image to a registry (optional)
5. Run a container from the image



Images and Stages



Dockerfile and Image

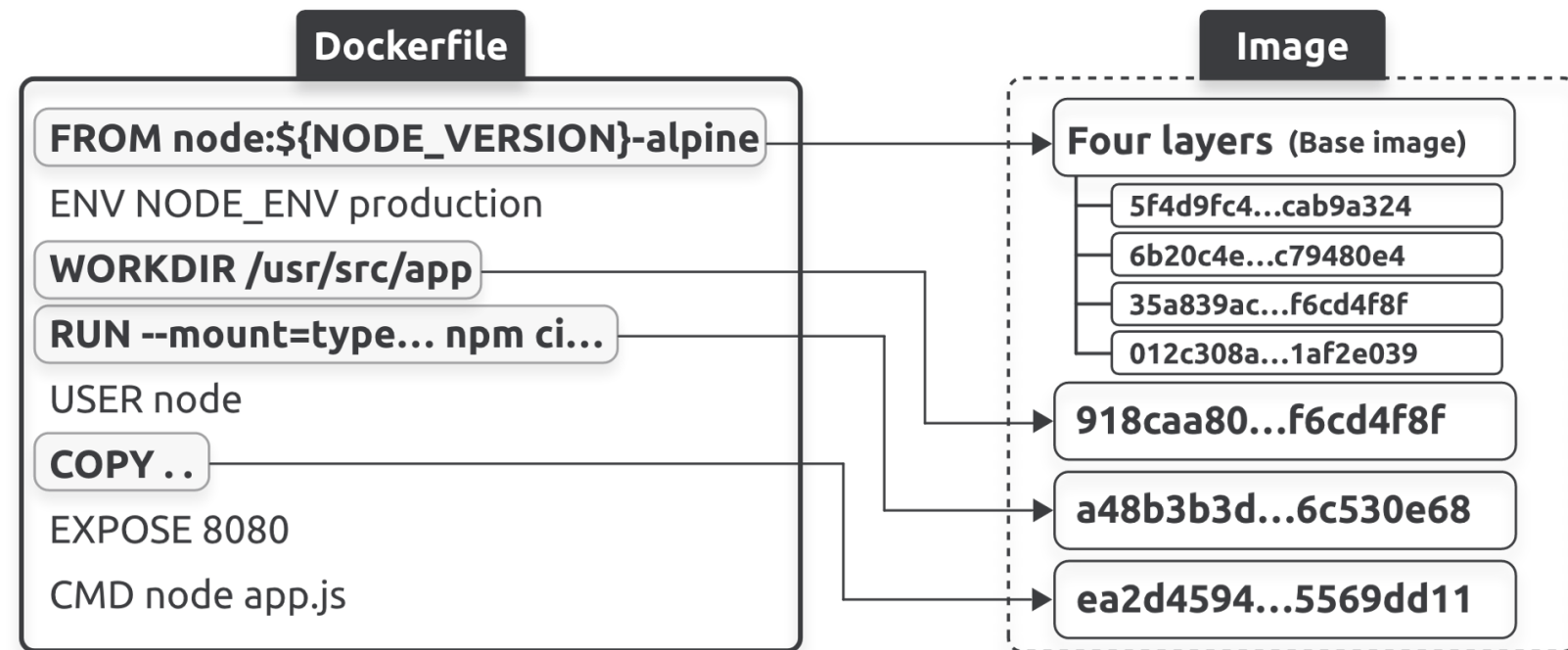
Creating Layers:

- FROM / RUN / COPY / WORKDIR

Metadata:

- EXPOSE / ENV / CMD / ENTRYPOINT

What can be a problem when using such Dockerfiles?



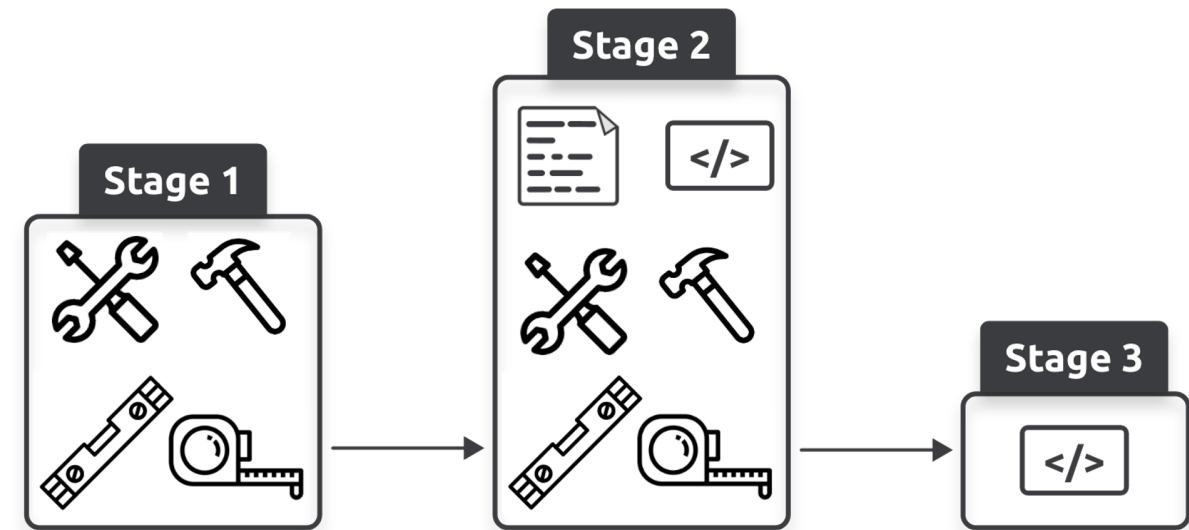
Dockerfile and Image

```
FROM golang:1.23.4-alpine AS base          <<---- Stage 0
WORKDIR /src
COPY go.mod go.sum .
RUN go mod download
COPY . .

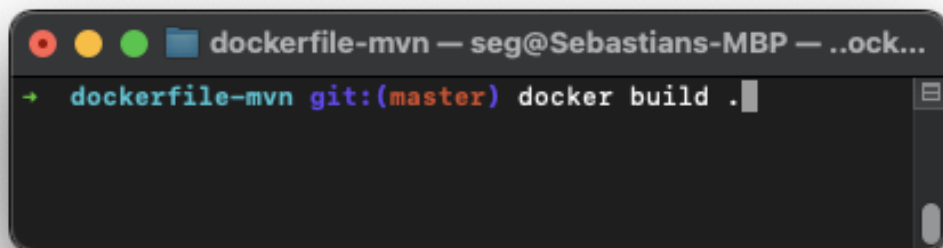
FROM base AS build-client                  <<---- Stage 1
RUN go build -o /bin/client ./cmd/client

FROM base AS build-server                  <<---- Stage 2
RUN go build -o /bin/server ./cmd/server

FROM scratch AS prod                       <<---- Stage 3
COPY --from=build-client /bin/client /bin/
COPY --from=build-server /bin/server /bin/
ENTRYPOINT [ "/bin/server" ]
```



How to build an image? – Full-fledged Image



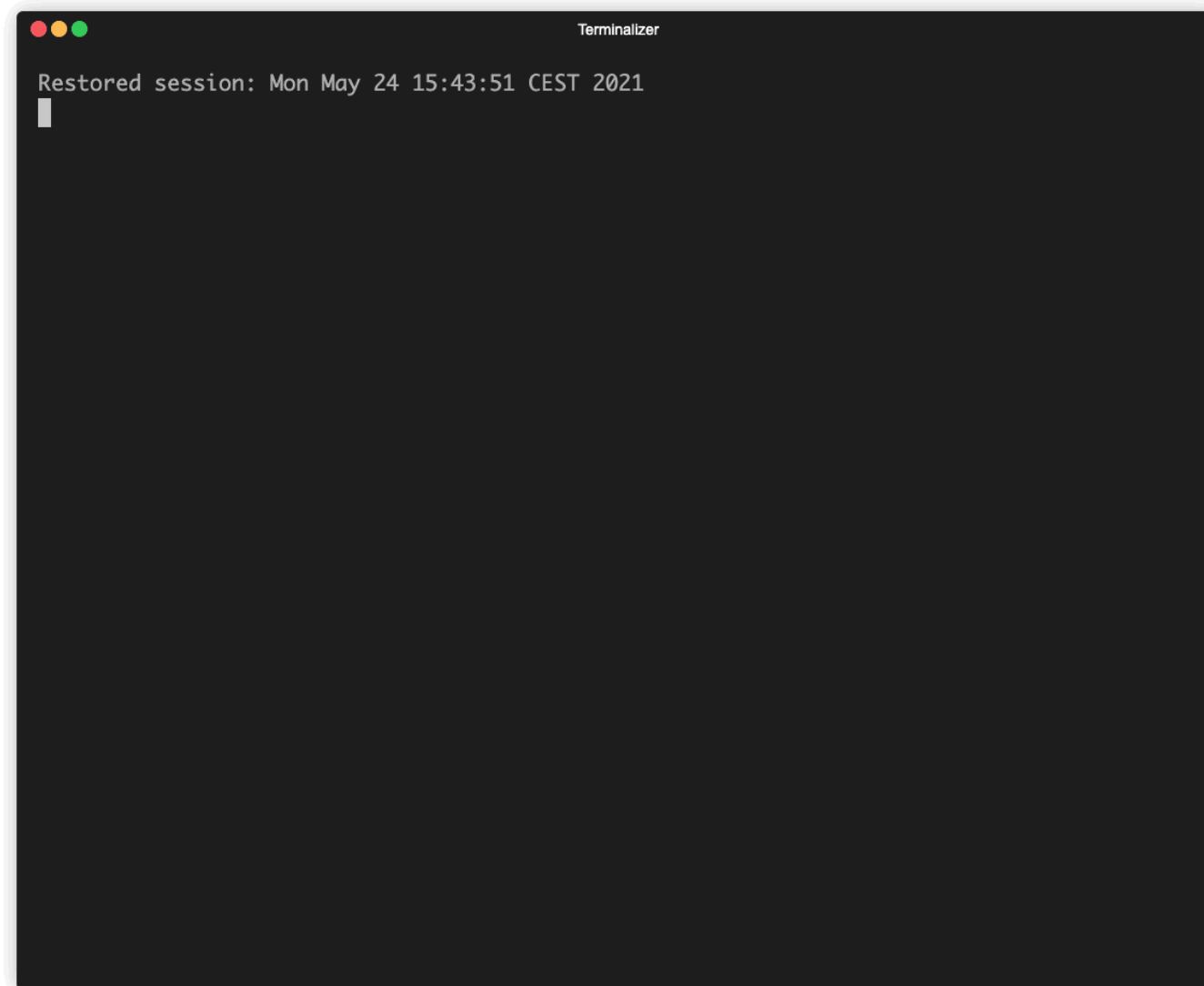
```
dockerfile-mvn — seg@Sebastians-MBP — ..ock...
→ dockerfile-mvn git:(master) docker build .
```

+:

- Easy to write and to maintain
- Clean structure and transparent content

-:

- Large image
- Large attack surface
- Not necessary tools for running



How to build an image? – Multistage Builds

+:

- Separate Build and Run-Images
- Smaller Run Images with optimal attack surface

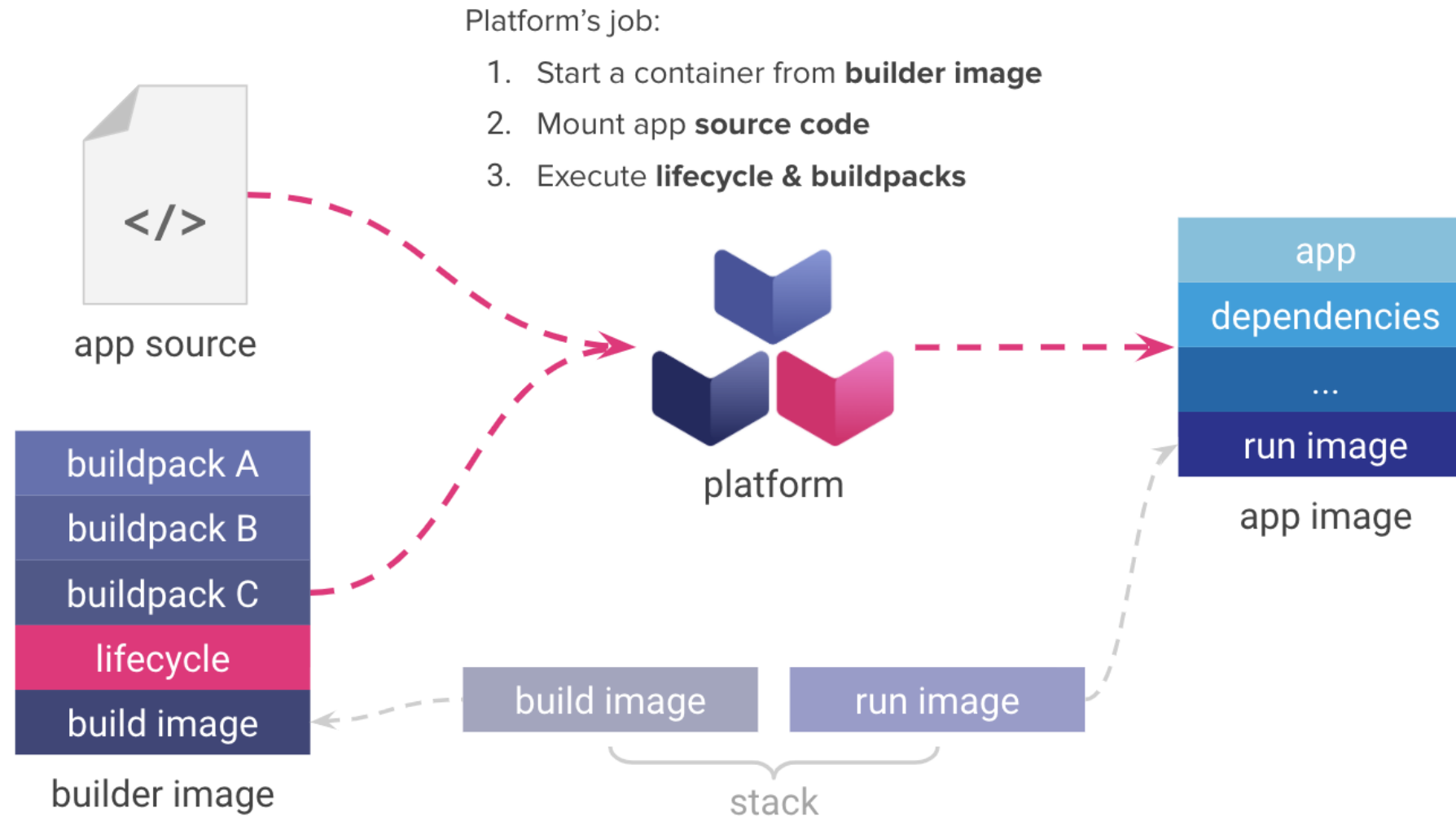
-:

- More complicate to develop
- Not as transparent in maintaining

```
Terminalizer
→ dockerfile-mvn git:(master) x
```

```
dockerfile-mvn — seg@Sebastians-MBP — ..ockerfile-mvn — zs...
→ dockerfile-mvn git:(master) docker build . -f Dockerfile-builder
```


How to build an image? – Build Packs



<https://tanzu.vmware.com/developer/guides/containers/cnb-what-is/>

<https://buildpacks.io/docs/concepts/>

How to build an image? – Build Packs

+:

- Dockerimage without Dockerfile
- Auto Detect of Frameworks

-:

- New project, bugs occur
- Complicated architecture, hard to track errors in framework

```
native-mvn — pack build imagename -B paketobuildpacks/builder:base -e "BP_JVM_VERSION=17" —...
→ native-mvn git:(master) × pack build imagename -B paketobuildpacks/builder:base -e "BP_JVM_VERSION=17"
-e "BP_MAVEN_BUILD_ARGUMENTS=-Dmaven.test.skip=true package"
base: Pulling from paketobuildpacks/builder
Digest: sha256:119ef259461025a58700d166c8bfccb333cd8464eace7d323161ae47ecc229bc
Status: Image is up to date for paketobuildpacks/builder:base
base-cnb: Pulling from paketobuildpacks/run
Digest: sha256:89feae79c197ad6988ae358c27af056cb414c34a405ef581d61037012d7d5a4f
Status: Image is up to date for paketobuildpacks/run:base-cnb
==> ANALYZING
Previous image with name "imagename" not found
==> DETECTING
9 of 24 buildpacks participating
paketo-buildpacks/ca-certificates 3.2.2
paketo-buildpacks/bellsoft-liberica 9.3.4
paketo-buildpacks/syft 1.11.1
paketo-buildpacks/maven 6.5.2
paketo-buildpacks/executable-jar 6.2.3
paketo-buildpacks/apache-tomcat 7.3.4
paketo-buildpacks/liberty 1.1.0
paketo-buildpacks/dist-zip 5.2.3
paketo-buildpacks/spring-boot 5.10.0
==> RESTORING
==> BUILDING
```

<https://paketo.io/docs/builders/>

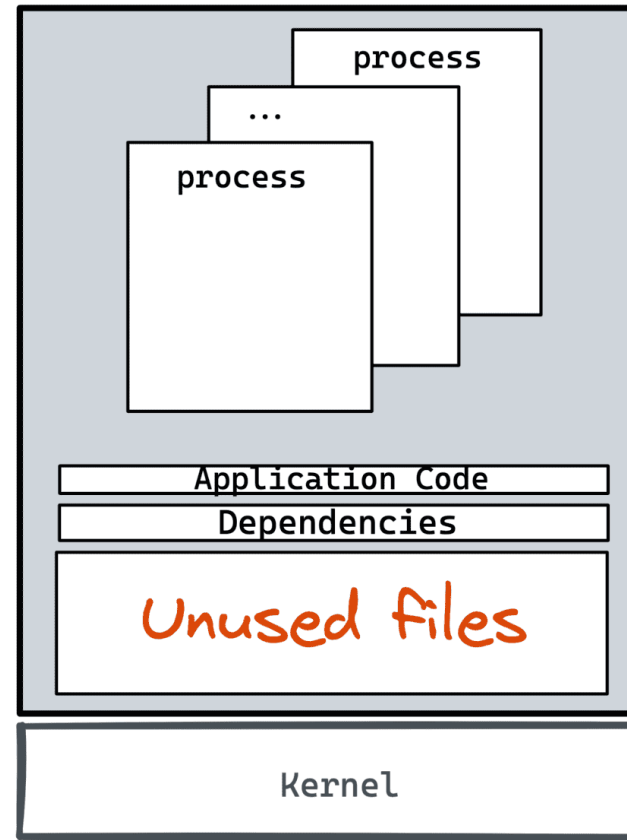
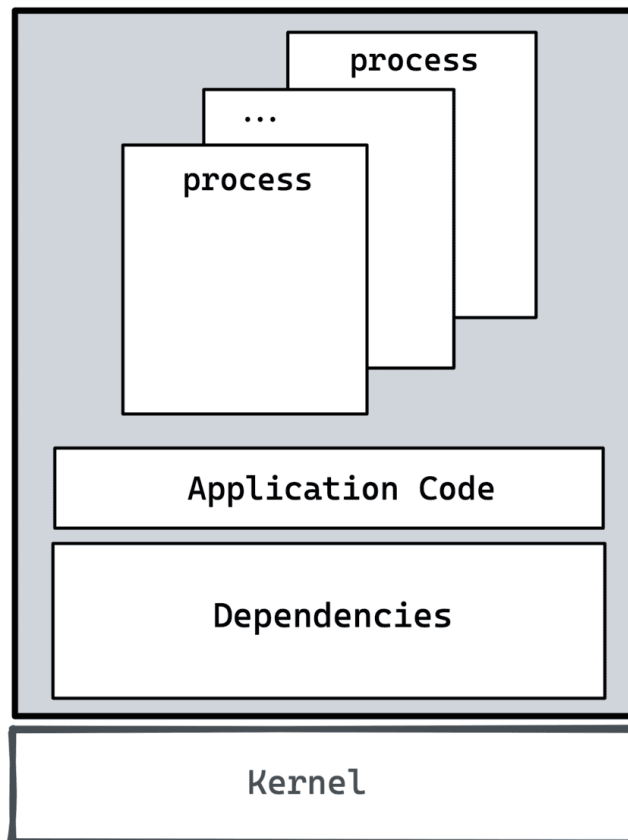
Choice of Base image

Containers

Theory

vs.

Practice



BaseImage:

Based on a distribution

- Alpine: small
- Distrioleless: no bash/distro really available
- Chainguard
<https://images.chainguard.dev/>
- Ubi
<https://catalog.redhat.com/software/base-images>

The Baseimage represents your attack surface as does the installed software/dependencies

<https://github.com/anchore/syft>

Generating SBOM - syft



SBOM:

- Components in the artifact
- List for licensing and security
- “Supply chain” for software

Be aware what you have in production.

```
seg — seg@Sebastians-MBP — ~ — zsh -l...  
→ ~ syft paketobuildpacks/builder:0.2.65-base
```

Security Scanning - gype

<https://github.com/anchore/gype>



```
seg — seg@Sebastians-MBP — ~ — zsh -l...  
→ ~ gype paketobuildpacks/builder:0.2.65-base
```

Best practices for building images

- Run only one process as PID 1 and register signal handlers
- Regular changing parts should be in the upper layers
- Remove unnecessary parts / packages
- No root-user / no writing (of possible)
- Build the smallest image possible
- Try to reuse common layers
- Scan your images
- Establish tagging strategy
- Be aware of public images



<https://cloud.google.com/blog/products/containers-kubernetes/7-best-practices-for-building-containers?hl=en>

